

在 Cloud Run 使用 Node.js 建構 Slack 機器人

來源：<https://codelabs.developers.google.com/codelabs/cloud-slack-bot?hl=zh-tw>

步驟數：13

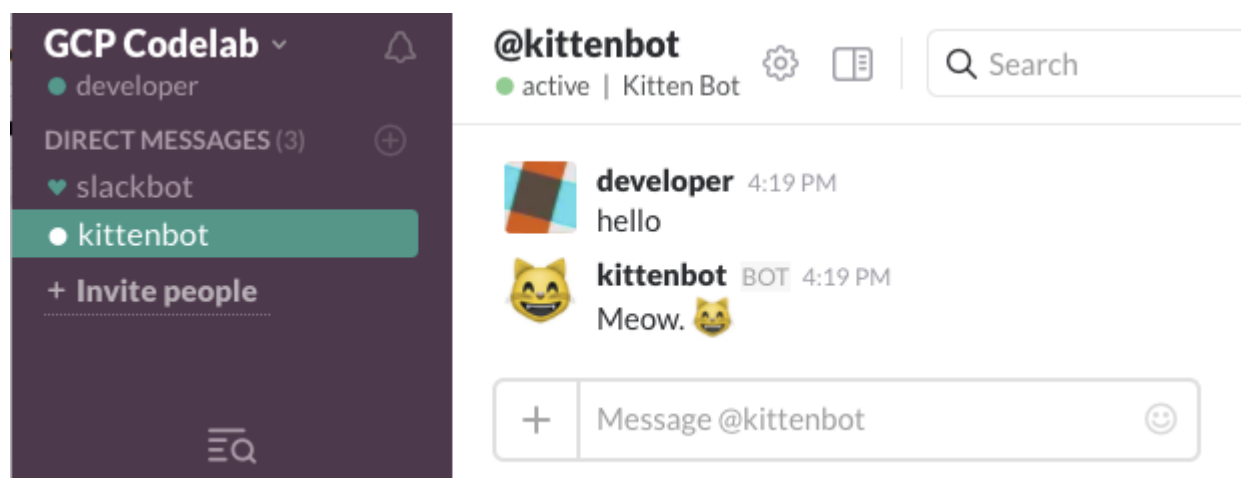
在這個程式碼研究室中，您將瞭解如何在 Google Cloud 中建構及執行 Slack 機器人。這項服務使用全代管運算平台 Cloud Run，可自動調整無狀態容器的資源配置。

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. 啟用 API](#)
4. [4. 建立 Slack 工作區](#)
5. [5. 建立 Slack 機器人使用者](#)
6. [6. 保護個人機密資訊](#)
7. [7. 取得範例程式碼](#)
8. [8. 部署應用程式](#)
9. [9. 啟用 Slack 事件](#)
10. [10. 測試機器人](#)
11. [11. 額外內容 - 更新機器人](#)
12. [12. 清除所用資源](#)
13. [13. 恭喜！](#)

步驟 1 · 約 0 分鐘

1. 總覽



在本程式碼研究室中，您將瞭解如何使用 [Botkit 工具包](#) 建構 [Slack 機器人](#)，並在 [Google Cloud](#) 上執行。你可以在 Slack 頻道中與機器人互動。

課程內容

- 如何在 Slack 中建立機器人自訂整合項目
- 如何使用 [Secret Manager](#) 保護 Slack 密鑰
- 如何在 [Cloud Run](#) 上部署 Slack 機器人。Cloud Run 是全代管運算平台，可自動調整無狀態容器的資源配置。

軟硬體需求

- 具備 Google Cloud 專案
 - [Chrome](#) 或 [Firefox](#) 瀏覽器
-

步驟 2 · 約 1 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button circled in blue. Below this, the 'Select a project' section includes a search bar and a 'NEW PROJECT' button circled in blue. The 'New Project' form is shown below, with the 'Project name' field containing 'my-kubernetes-codelab' and the 'Project ID' field containing 'my-kubernetes-codelab', which is circled in blue. The 'Location' field is set to 'No organization'. At the bottom, there are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

雖然您可以透過筆電遠端操作 Google Cloud，但在本教學課程中，您將使用 [Cloud Shell](#)，這是 Cloud 中執行的指令列環境。

啟用 Cloud Shell

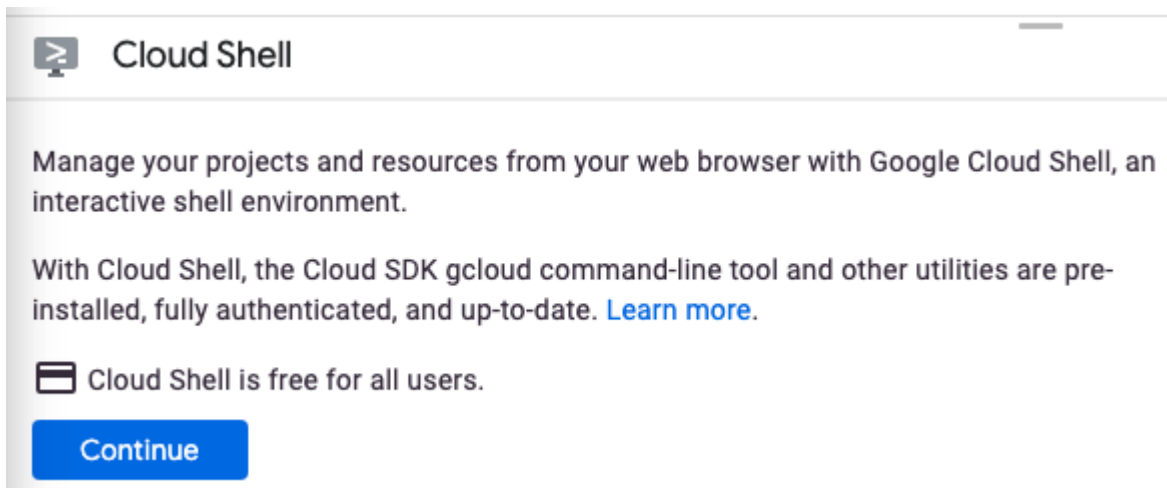
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



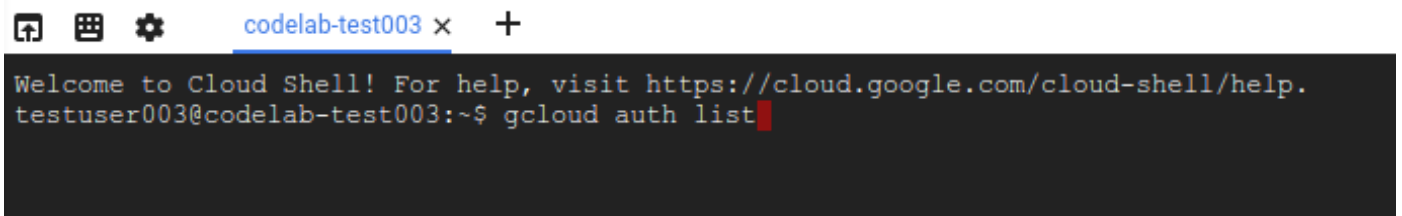
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

Credentialed Accounts

ACTIVE ACCOUNT

```
* <my_account>@<my_domain.com>
```

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]  
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 1 分鐘

3. 啟用 API

在 Cloud Shell 中，啟用 Artifact Registry、Cloud Build、Cloud Run 和 Secret Manager API：

```
gcloud services enable \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com \  
  run.googleapis.com \  
  secretmanager.googleapis.com
```

這會輸出類似以下的成功訊息：

```
Operation "operations/..." finished successfully.
```

在幕後，Cloud Run 會與容器生態系統配對，以部署應用程式：

- [Cloud Build](#) 會自動從原始碼建構容器映像檔，並推送至 Artifact Registry。
- [Artifact Registry](#) 會管理容器映像檔。

現在，您可以準備及部署應用程式了...

步驟 4 · 約 1 分鐘

4. 建立 Slack 工作區

您必須擁有可建立自訂整合的 Slack 工作區。如果您沒有要用於本教學課程的工作區，可以[免費建立工作區](#)。



First, enter your email

Just one more email — a quick confirmation — before you say goodbye to overstuffed inboxes for good.

Your email address

Next →



步驟 5 · 約 1 分鐘

5. 建立 Slack 機器人使用者

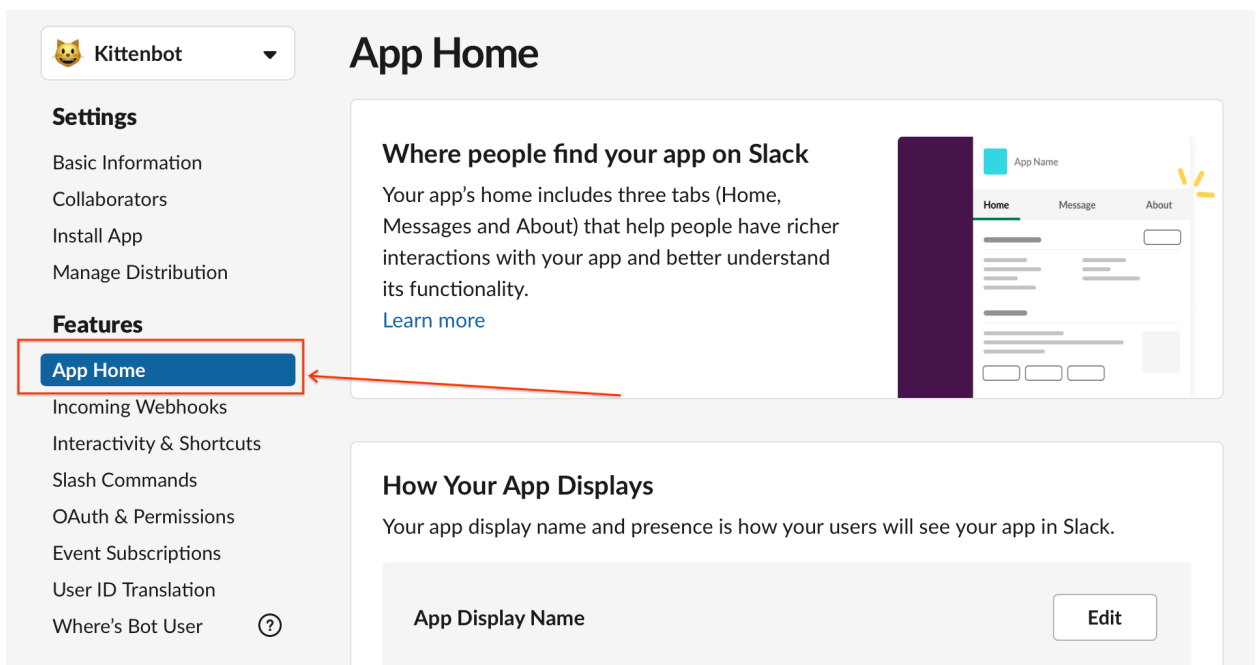
機器人使用者可以監聽 Slack 上的訊息、張貼訊息及上傳檔案。在本程式碼研究室中，您將建立機器人，發布簡單的問候訊息。

建立新的 Slack 應用程式

- 前往 [Slack 應用程式管理頁面](#)。
- 按一下右上角的「建立新應用程式」按鈕。
- 為應用程式命名，例如「Kittenbot」。
- 選擇要安裝的 Slack 團隊。

建立機器人使用者

- 前往左側面板「功能」下方的「應用程式首頁」



- 按一下「Review Scopes to Add」，為機器人權杖指派範圍
- 向下捲動至「機器人權杖範圍」，然後點選「新增 OAuth 範圍」。選取「以 Kittenbot 身分傳送訊息」 `chat:write`

Scopes

A Slack app's capabilities and permissions are governed by the [scopes](#) it requests.

Bot Token Scopes

Scopes that govern what your app can access.

OAuth Scope	Description	
chat:write	Send messages as Kittenbot	
Add an OAuth Scope		

- 向上捲動並按一下「Install App to your Workspace」(在工作區安裝應用程式) 按鈕。

- 這會將應用程式安裝到團隊、新增您剛建立的機器人使用者，並產生機器人權杖。
- 系統提示時，請按一下「允許」，授權機器人在工作區中對話。

啟用訊息和指令

- 向下捲動至「顯示分頁」，並確認已啟用下列兩個選項：

Messages Tab

Direct messages your app sends will show in this tab.



Allow users to send Slash commands and messages from the messages tab

取得用戶端簽署密鑰

- 前往「設定」下方的「基本資訊」。
- 向下捲動至「簽署密鑰」，按一下「顯示」，然後將密鑰複製到剪貼簿：

Client Secret

.....

Show

Regenerate

You'll need to send this secret along with your client ID when making your [oauth.v2.access](#) request.

Signing Secret

.....

Show

Regenerate

Slack signs the requests we send you using this secret. Confirm that each request comes from Slack by verifying its unique signature.

- 將密鑰儲存在環境變數中：

```
CLIENT_SIGNING_SECRET=PASTE_THE_SIGNING_SECRET
```

取得機器人權杖

- 前往「功能」下方的「OAuth 和權限」。
- 按一下「複製」按鈕，將「機器人使用者 OAuth 存取權杖」文字複製到剪貼簿。

OAuth Tokens for Your Workspace

These tokens were automatically generated when you installed the app to your team. You can use these to authenticate your app. [Learn more.](#)

Bot User OAuth Token

BOT_TOKEN_TO_COPY

Copy

Access Level: Workspace

- 將機器人權杖儲存在環境變數中：

BOT_TOKEN=PASTE_THE_BOT_TOKEN

別擔心，如需再次取得這些權杖，可以從 [應用程式管理頁面](#) 返回這個設定頁面。

請**謹慎**使用機器人使用者 OAuth 存取權杖和簽署密鑰。請將這些權杖視為其他私密權杖，請勿將權杖儲存在版本控管系統中，也不要公開分享。

6. 保護個人機密資訊

我們希望確保機器人權杖和用戶端簽署密鑰安全無虞。如果將這些密鑰硬式編碼到原始碼中，很可能會將密鑰發布到版本控制系統，或嵌入 Docker 映像檔，導致密鑰意外曝光。

[Secret Manager](#) 提供安全又便利的方法，可儲存 API 金鑰、密碼、憑證和其他機密資料。Secret Manager 提供單一可靠的資料來源，可讓您集中管理、存取及稽核 Google Cloud 中的密鑰。

建立密鑰

使用下列指令儲存用戶端簽署密鑰和機器人權杖：

- 用戶端簽署密鑰

```
echo -n $CLIENT_SIGNING_SECRET | gcloud secrets create client-signing-secret \
  --replication-policy automatic \
  --data-file -
```

- 機器人權杖

```
echo -n $BOT_TOKEN | gcloud secrets create bot-token \
  --replication-policy automatic \
  --data-file -
```

存取密鑰

請確認密鑰是否已正確建立，以及權限是否正常運作。使用下列指令存取密鑰：

```
echo $(gcloud secrets versions access 1 --secret client-signing-secret)
echo $(gcloud secrets versions access 1 --secret bot-token)
```

您也可以在 [Google Cloud 控制台](#) 中查看及管理密鑰。

步驟 7 · 約 2 分鐘

7. 取得範例程式碼

在 Cloud Shell 的指令列中，執行下列指令來複製 [GitHub 存放區](#)：

```
git clone https://github.com/googlecodecloudlabs/cloud-slack-bot.git
```

將目錄變更為 `cloud-slack-bot/start`。

```
cd cloud-slack-bot/start
```

瞭解程式碼

使用偏好的指令列編輯器 (nano、vim、emacs...) 開啟 `kittenbot.js` 檔案，或使用下列指令直接在 Cloud Shell 編輯器中開啟目前的資料夾：

```
cloudshell workspace .
```

小貓機器人程式碼有兩項主要功能。一個是擷取密鑰，另一個則是執行機器人。

首先，匯入依附元件：

kittenbot.js

```
const { Botkit } = require('botkit');
const {
  SlackAdapter,
  SlackEventMiddleware,
} = require('botbuilder-adapters-slack');
const { SecretManagerServiceClient } = require('@google-cloud/secret-manager');
```

`SlackAdapter` 和 `SlackEventMiddleware` 是擴充 `Botkit` 的套件，可讓機器人輕鬆將訊息轉換為 Slack API 格式，或從 Slack API 格式轉換。您可以使用 `Secret Manager` 用戶端，存取先前步驟中儲存的密鑰。

接著是擷取 `Secret` 的函式：

```

/**
 * Returns the secret string from Google Cloud Secret Manager
 * @param {string} name The name of the secret.
 * @return {Promise<string>} The string value of the secret.
 */
async function accessSecretVersion(name) {
  const client = new SecretManagerServiceClient();
  const projectId = process.env.PROJECT_ID;
  const [version] = await client.accessSecretVersion({
    name: `projects/${projectId}/secrets/${name}/versions/1`,
  });

  // Extract the payload as a string.
  const payload = version.payload.data.toString('utf8');

  return payload;
}

```

這個函式會傳回驗證機器人所需的密鑰字串值。

下一個函式會初始化機器人：

```

/**
 * Function to initialize kittenbot.
 */
async function kittenbotInit() {
  const adapter = new SlackAdapter({
    clientSigningSecret: await accessSecretVersion('client-signing-secret'),
    botToken: await accessSecretVersion('bot-token'),
  });

  adapter.use(new SlackEventMiddleware());

  const controller = new Botkit({
    webhook_uri: '/api/messages',
    adapter: adapter,
  });

  controller.ready(() => {
    controller.hears(
      ['hello', 'hi', 'hey'],
      ['message', 'direct_message'],
      async (bot, message) => {
        await bot.reply(message, 'Meow. :smile_cat:');
      }
    );
  });
}

```

函式的第一部分會使用密鑰設定 SlackAdapter，然後指定接收訊息的端點。然後，當控制器開啟時，機器人會回覆任何包含「hello」、「hi」或「hey」的訊息，回覆內容為「Meow. 🐱」。

請查看應用程式資訊清單中的下列特定部分：

package.json

```
{
  // ...
  "scripts": {
    "start": "node kittenbot.js",
    // ...
  },
  "engines": {
    "node": "16"
  },
  // ...
}
```

您可以使用 Cloud Run，直接從原始碼部署 Node.js 應用程式。系統會執行下列作業：

- Cloud Run 會呼叫 Cloud Build 來建構容器映像檔 (請參閱「[從原始碼部署](#)」)。
- 如果原始碼目錄中存在 `Dockerfile`，Cloud Build 會使用該檔案建構容器映像檔。
- 如果不是，Cloud Build 會呼叫 [Buildpacks](#) 分析來源，並自動產生可供正式環境使用的映像檔。
- 建構套件會偵測 `package.json` 資訊清單，並建構 Node.js 映像檔。
- `scripts.start` 欄位會決定應用程式的啟動方式。
- `engines.node` 欄位會決定容器基本映像檔的 Node.js 版本。
- 部署時，系統會自動套用已知的安全性修正。

現在可以部署應用程式了！

步驟 8 · 約 3 分鐘

8. 部署應用程式

Slack Events API 會使用 Webhook 傳送有關事件的外寄訊息。設定 Slack 應用程式時，您必須提供 Slack API 可 Ping 的公開存取網址。

Cloud Run 是託管 Webhook 目標的絕佳解決方案。您可以隨意使用任何語言或執行階段，而且這項服務提供並行處理功能，因此應用程式可以處理的資料量會大幅增加。

擷取專案 ID

定義 PROJECT_ID 環境變數：

```
PROJECT_ID=$(gcloud config get-value core/project)
```

定義 Cloud Run 區域

Cloud Run 具有「地區性」，這表示執行 Cloud Run 服務的基礎架構位於特定地區，並由 Google 代管，可為該地區內所有區域提供備援功能。定義要用於部署作業的區域，例如：

```
REGION="us-central1"
```

如需目前支援的區域清單，請參閱「[Cloud Run \(全代管\) 地區](#)」。

更新權限

如要存取 Secret Manager 中的密碼，必須授予 Cloud Run 服務帳戶 `roles/secretmanager.secretAccessor` 角色。

首先，將預設服務帳戶儲存至環境變數：

```
SERVICE_ACCOUNT=$(gcloud iam service-accounts list \
  --format "value(email)" \
  --filter "displayName:Compute Engine default service account")
```

確認你已儲存電子郵件地址：

```
echo $SERVICE_ACCOUNT
```

服務帳戶的格式如下：`PROJECT_NUMBER-compute@developer.gserviceaccount.com`。

取得電子郵件地址後，請為服務帳戶啟用角色：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:$SERVICE_ACCOUNT \
  --role roles/secretmanager.secretAccessor
```


部署應用程式

Cloud Run 服務會公開一個不重複的端點，並自動調整底層基礎架構的資源配置來處理傳入要求。

將應用程式部署至 Cloud Run：

```
gcloud run deploy kittenbot \  
  --source . \  
  --platform managed \  
  --region $REGION \  
  --set-env-vars PROJECT_ID=$PROJECT_ID \  
  --allow-unauthenticated
```

- 這會建立名為 `kittenbot` 的服務。
- `--source` 選項會使用目前資料夾，透過 Cloud Build 建構應用程式。Cloud Build 會自動偵測 `package.json` 檔案是否存在。
- 您也可以使用下列指令定義預設區域：`gcloud config set run/region $REGION`
- 您也可以使用下列指令，將 Cloud Run 設為預設代管：`gcloud config set run/platform managed`
- `--set-env-vars` 選項會設定服務環境變數。
- `--allow-unauthenticated` 選項會將服務設為公開。

如要查看所有選項，請使用 `gcloud run deploy --help`。

第一次執行這項操作時，系統會提示您建立 Artifact Registry 存放區。輕觸 Enter 鍵進行驗證：

```
Deploying from source requires an Artifact Registry Docker repository to store  
built containers. A repository named [cloud-run-source-deploy] in region [REGION]  
will be created.
```

```
Do you want to continue (Y/n)?
```

這會啟動來源程式碼的上傳作業，將程式碼上傳至 Artifact Registry 存放區，並建構容器映像檔：

```
Building using Dockerfile and deploying container ...  
* Building and deploying new service... Building Container.  
OK Creating Container Repository...  
OK Uploading sources...  
* Building Container... Logs are available at ...
```

然後稍等片刻，等待建構及部署作業完成。部署成功之後，指令列會顯示服務網址：

```
...
OK Building and deploying new service... Done.
OK Creating Container Repository...
OK Uploading sources...
OK Building Container... Logs are available at ...
OK Creating Revision... Creating Service.
OK Routing traffic...
OK Setting IAM Policy...
Done.
Service [SERVICE]... has been deployed and is serving 100 percent of traffic.
Service URL: https://SERVICE-PROJECTHASH-REGIONID.a.run.app
```

您可以使用下列指令取得服務網址：

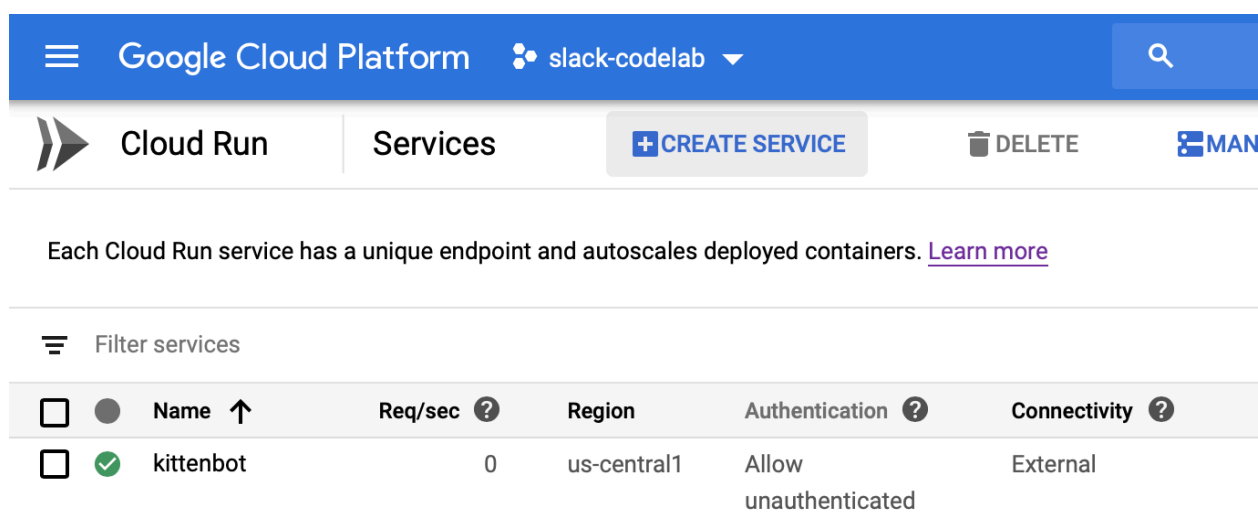
```
SERVICE_URL=$( \
  gcloud run services describe kittenbot \
  --platform managed \
  --region $REGION \
  --format "value(status.url)" \
)
echo $SERVICE_URL
```

網址格式如下：

```
https://kittenbot-PROJECTHASH-REGIONID.a.run.app
```

這個網址將做為啟用 Slack Events API 的基礎。複製到剪貼簿，以便在下一步中使用。

您的服務現已上線，可供大眾使用！詳情請參閱 [Cloud Run 控制台](#)。



The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo, a user profile icon labeled 'slack-codelab', and a search icon. Below the header, there's a navigation bar with 'Cloud Run' selected, followed by 'Services', a '+ CREATE SERVICE' button, a 'DELETE' button, and a 'MAN' button. The main content area has a heading 'Each Cloud Run service has a unique endpoint and autoscales deployed containers. [Learn more](#)'. Below this, there's a 'Filter services' section. A table lists the services:

☐	☒	Name ↑	Req/sec ?	Region	Authentication ?	Connectivity ?
☐	☒	kittenbot	0	us-central1	Allow unauthenticated	External

您可以查看上次修訂版本建立時間、收到的流量，以及查看記錄。如果我們點選記錄，會看到 Botkit 控制器已開啟，並準備接收訊息。

現在開始從 Slack 頻道傳送訊息吧！

步驟 9 · 約 2 分鐘

9. 啟用 Slack 事件

如先前所述，kittenbot 程式碼會為 Webhook 目標指定相對端點。

kittenbot.js

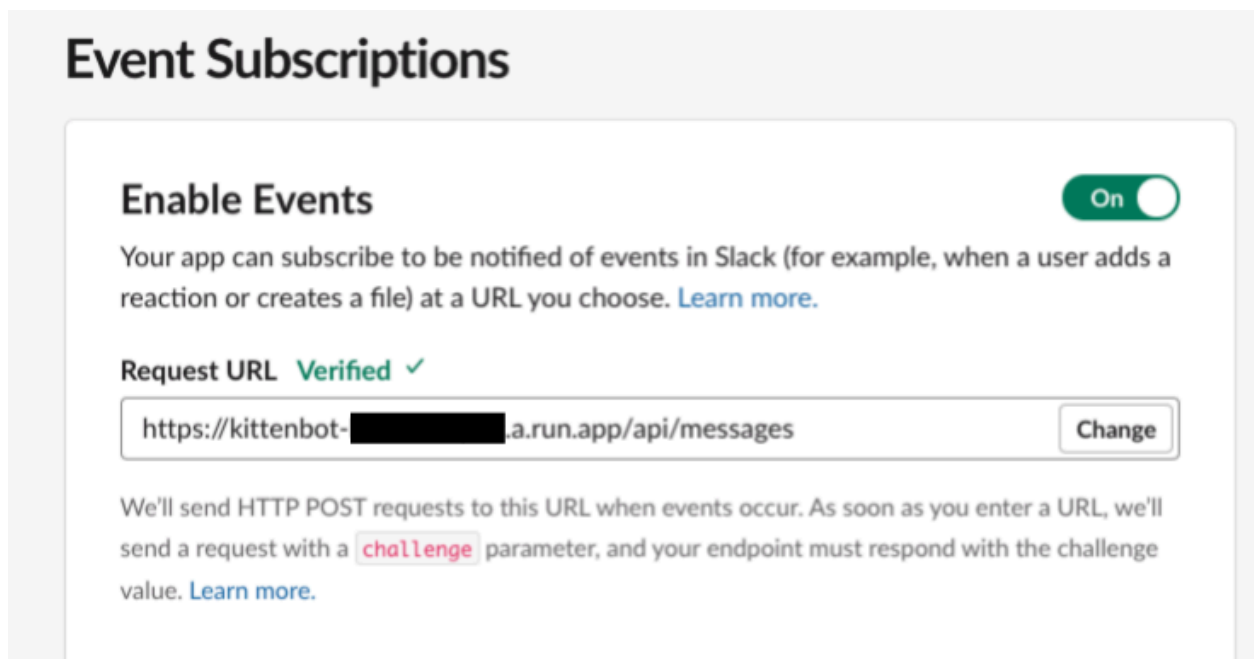
```
const controller = new Botkit({
  webhook_uri: '/api/messages',
  adapter: adapter,
});
```

也就是說，完整網址會是 Cloud Run 服務的基本部分，加上 `/api/messages`。

啟用活動

在[應用程式管理頁面](#)中，前往側欄的「Events Subscriptions」部分，然後開啟「Enable Events」。輸入服務網址：

```
PASTE_THE_SERVICE_URL/api/messages
```



Event Subscriptions

Enable Events On

Your app can subscribe to be notified of events in Slack (for example, when a user adds a reaction or creates a file) at a URL you choose. [Learn more.](#)

Request URL Verified ✓

Change

We'll send HTTP POST requests to this URL when events occur. As soon as you enter a URL, we'll send a request with a **challenge** parameter, and your endpoint must respond with the challenge value. [Learn more.](#)

系統可能會在你輸入網址的過程中嘗試驗證，如果失敗，請按一下「重試」。

訂閱

訂閱所有訊息機器人事件。

Subscribe to bot events



Apps can subscribe to receive events the bot user has access to (like new messages in a channel). If you add an event here, we'll add the necessary [OAuth scope](#) for you.

Event Name	Description	Required Scope	
message.im	A message was posted in a direct message channel	im:history	
message.groups	A message was posted to a private channel	groups:history	
message.channels	A message was posted to a channel	channels:history	
message.mpim	A message was posted in a multiparty direct message channel	mpim:history	

Add Bot User Event

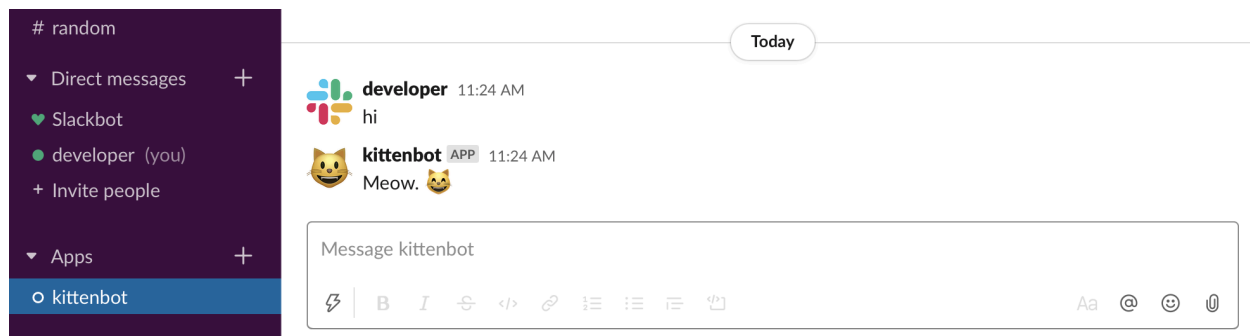
按一下頁面底部的「儲存變更」。系統會提示你重新安裝應用程式。請按照提示操作，然後點選「允許」。

此時，機器人已完全整合！工作區中的訊息會觸發 Slack 將訊息傳送至 Cloud Run 服務，而該服務會以簡單問候訊息做為回應。

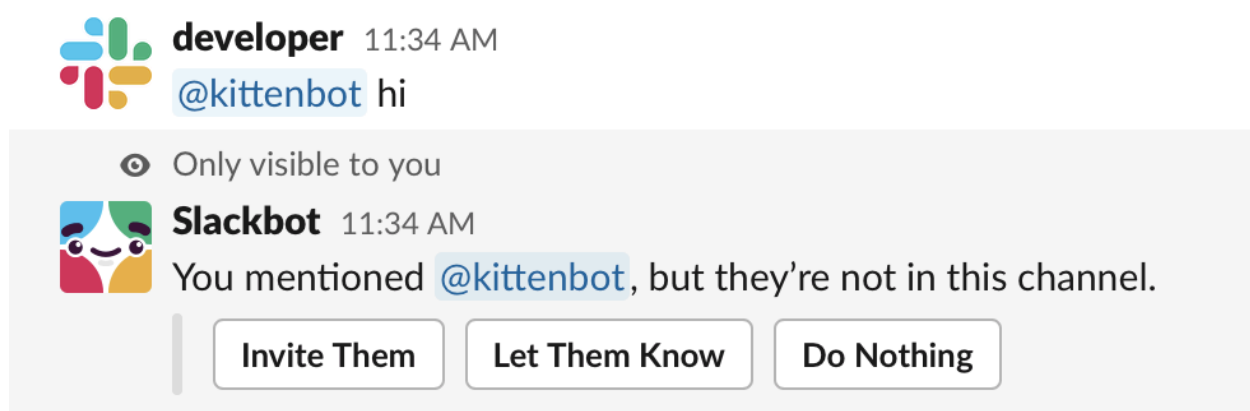
步驟 10 · 約 1 分鐘

10. 測試機器人

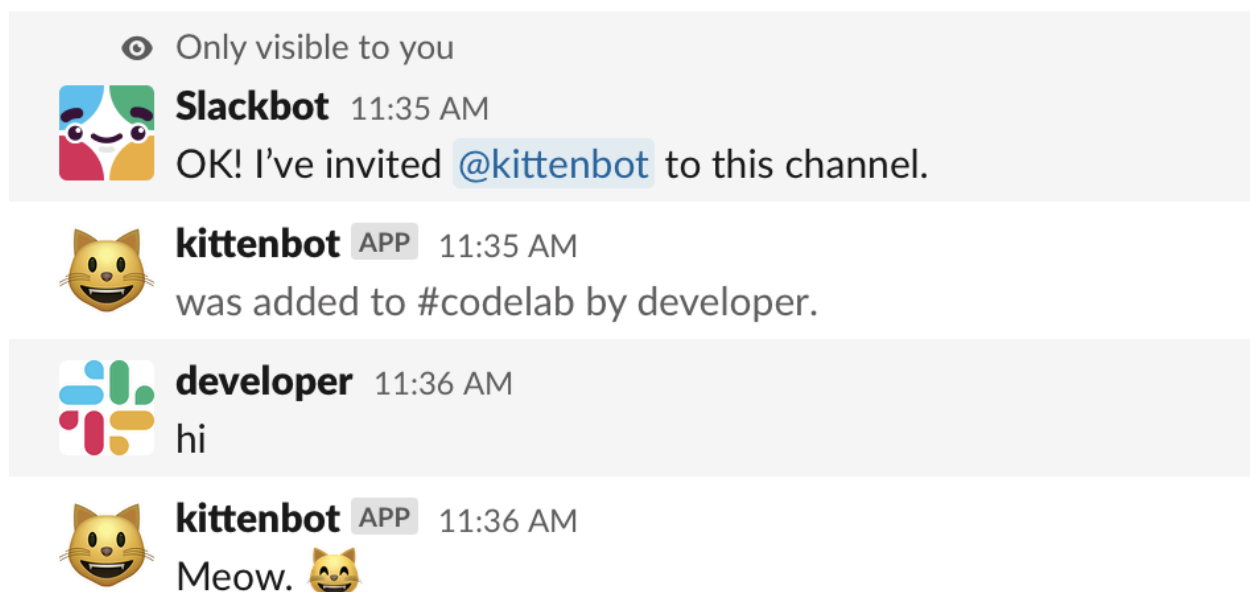
傳送即時訊息給 Kittenbot：



輸入「@kittenbot」，然後按一下「邀請他們」，即可將 kittenbot 新增至頻道：



現在頻道中的所有人都能與 Kittenbot 互動！



Slack 中的每則訊息都會觸發事件，並將 HTTP POST 訊息傳送至 Cloud Run 服務。查看 Cloud Run 服務記錄時，您會發現每則訊息都對應記錄中的 POST 項目。

▶		2020-03-24 17:13:51.108	PDT	POST	200	264 B	2.7 s	Slackbot ...
▶		2020-03-24 17:14:30.261	PDT	POST	200	259 B	188 ms	Slackbot ...
▶		2020-03-24 17:14:31.523	PDT	POST	200	259 B	171 ms	Slackbot ...
▶		2020-03-24 17:14:32.108	PDT	POST	200	257 B	9 ms	Slackbot ...
▶		2020-03-24 17:14:42.267	PDT	POST	200	259 B	163 ms	Slackbot ...
▶		2020-03-24 17:14:43.012	PDT	POST	200	257 B	9 ms	Slackbot ...

Kittenbot 會以「Meow. 🐱」。

想要更像對話的機器人嗎？繼續進行下一個加分步驟，更新機器人。

11. 額外內容 - 更新機器人

這個選用部分大約需要幾分鐘。您可以直接跳到「清理」步驟。

對話討論串

我們希望機器人能做的遠不只說「喵」。但要如何部署在 Cloud Run 上執行的項目新版本？

將目錄變更為 `cloud-slack-bot/extra-credit`：

```
cd ../extra-credit/
```

在 Cloud Shell 編輯器中開啟目前的資料夾：

```
cloudshell workspace .
```

Botkit 提供[處理對話](#)的功能，有了這些功能，機器人就能要求更多資訊，並對訊息做出回應，而不只是回覆單字。

定義對話方塊

首先，請查看檔案結尾的對話函式定義：


```

// ...
const maxCats = 20;
const catEmojis = [
  ':smile_cat:',
  ':smiley_cat:',
  ':joy_cat:',
  ':heart_eyes_cat:',
  ':smirk_cat:',
  ':kissing_cat:',
  ':scream_cat:',
  ':crying_cat_face:',
  ':pouting_cat:',
  ':cat:',
  ':cat2:',
  ':leopard:',
  ':lion_face:',
  ':tiger:',
  ':tiger2:',
];

/**
 * Function to concatenate cat emojis
 * @param {number} numCats Number of cat emojis.
 * @return {string} The string message of cat emojis.
 */
function makeCatMessage(numCats) {
  let catMessage = '';
  for (let i = 0; i < numCats; i++) {
    // Append a random cat from the list
    catMessage += catEmojis[Math.floor(Math.random() * catEmojis.length)];
  }
  return catMessage;
}

/**
 * Function to create the kitten conversation
 * @param {Object} controller The botkit controller.
 * @return {Object} The BotkitConversation object.
 */
function createKittenDialog(controller) {
  const convo = new BotkitConversation('kitten-delivery', controller);

  convo.ask('Does someone need a kitten delivery?', [
    {
      pattern: 'yes',
      handler: async (response, convo, bot) => {
        await convo.gotoThread('yes_kittens');
      },
    },
    {
      pattern: 'no',
      handler: async (response, convo, bot) => {

```

```

    await convo.gotoThread('no_kittens');
  },
},
{
  default: true,
  handler: async (response, convo, bot) => {
    await convo.gotoThread('default');
  },
},
]);

convo.addQuestion(
  'How many would you like?',
  [
    {
      pattern: '^[0-9]+?',
      handler: async (response, convo, bot, message) => {
        const numCats = parseInt(response);
        if (numCats > maxCats) {
          await convo.gotoThread('too_many');
        } else {
          convo.setVar('full_cat_message', makeCatMessage(numCats));
          await convo.gotoThread('cat_message');
        }
      },
    },
    {
      default: true,
      handler: async (response, convo, bot, message) => {
        if (response) {
          await convo.gotoThread('ask_again');
        } else {
          // The response '0' is interpreted as null
          await convo.gotoThread('zero_kittens');
        }
      },
    },
  ],
  'num_kittens',
  'yes_kittens'
);

// If numCats is too large, jump to start of the yes_kittens thread
convo.addMessage(
  'Sorry, {{vars.num_kittens}} is too many cats. Pick a smaller number.',
  'too_many'
);

convo.addAction('yes_kittens', 'too_many');

// If response is not a number, jump to start of the yes_kittens thread
convo.addMessage("Sorry I didn't understand that", 'ask_again');
convo.addAction('yes_kittens', 'ask_again');

```

```

// If numCats is 0, send a dog instead
convo.addMessage(
  {
    text:
      'Sorry to hear you want zero kittens. ' +
      'Here is a dog, instead. :dog:',
    attachments: [
      {
        fallback: 'Chihuahua Bubbles - https://youtu.be/s84dBopsIe4',
        text: '<https://youtu.be/s84dBopsIe4|' + 'Chihuahua Bubbles>!',
      },
    ],
  },
  'zero_kittens'
);

// Send cat message
convo.addMessage('{{vars.full_cat_message}}', 'cat_message');

convo.addMessage('Perhaps later.', 'no_kittens');

return convo;
}

```

這項新對話會根據回覆內容引導討論串。舉例來說，如果使用者對小貓問題的回覆是「否」，系統就會跳到標示為「no_kittens」的訊息，也就是該對話討論串的結尾。

將對話方塊新增至控制器

定義對話後，請參閱下列說明，瞭解如何將對話新增至控制器：

```

async function kittenbotInit() {
  // ...
  const controller = new Botkit({
    webhook_uri: '/api/messages',
    adapter: adapter,
  });

  // Add Kitten Dialog
  const convo = createKittenDialog(controller);
  controller.addDialog(convo);

  // Controller is ready
  controller.ready(() => {
    // ...
  });
}

```

觸發對話方塊

現在控制器可以使用這個對話方塊，請參閱當聊天機器人聽到「小貓」、「小貓們」、「貓」或「貓們」時，對話如何開始：

```
// ...

controller.ready(() => {
  controller.hears(
    ['hello', 'hi', 'hey'],
    ['message', 'direct_message'],
    async (bot, message) => {
      await bot.reply(message, 'Meow. :smile_cat:');
      return;
    }
  );

  // START: listen for cat emoji delivery
  controller.hears(
    ['cat', 'cats', 'kitten', 'kittens'],
    ['message', 'direct_message'],
    async (bot, message) => {
      // Don't respond to self
      if (message.bot_id !== message.user) {
        await bot.startConversationInChannel(message.channel, message.user);
        await bot.beginDialog('kitten-delivery');
        return;
      }
    }
  );
  // END: listen for cat emoji delivery

  // ...
});

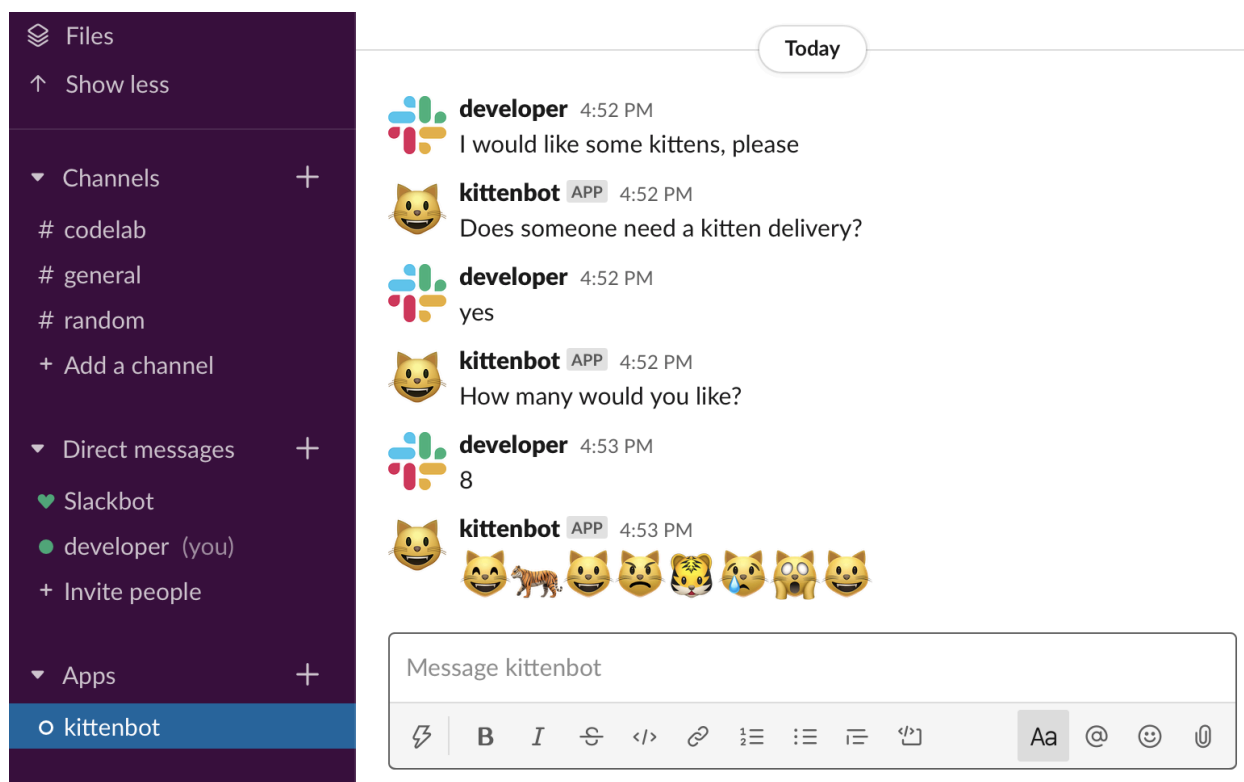
// ...
```

更新應用程式

將應用程式重新部署至 Cloud Run：

```
gcloud run deploy kittenbot \
  --source . \
  --platform managed \
  --region $REGION \
  --set-env-vars PROJECT_ID=$PROJECT_ID \
  --allow-unauthenticated
```

馬上試試



恭喜！您剛才將在 Cloud Run 上執行的 Slack 機器人更新為新版本。

斜線指令

如果不想與使用者對話，該怎麼做？如果只想透過簡單的指令觸發動作，該怎麼做？

Slack 提供這項功能，使用者只要在訊息方塊中輸入指令，就能透過斜線指令叫用應用程式。

啟用 Slack 斜線指令

- 前往應用程式管理頁面「功能」下方的「斜線指令」部分。
- 按一下「建立新指令」。
- 使用 kittenbot 服務網址設定 `/cats` 指令。請記得使用啟用 Events API 時的端點！這是您的網址，加上 `'/api/messages'`。

Edit Command

Command

/cats



Request URL

https://kittenbot-

a.run...



Short Description

Cat Emojis

Usage Hint

Tell us how many cats you want to recei...

Optionally list any parameters that can be passed.

Escape channels, users, and links sent to your app



Unescaped: @user #general

Preview of Autocomplete Entry

Commands matching "cats"

Kittenbot

/cats Tell us how many cats you want to receive -... Cat Emojis

+

/cats

- 按照提示更新應用程式和權限。

在控制器中新增斜線指令

請參閱控制器 `ready` 函式內新增斜線指令處理常式的方式：

```
// ...

// Controller is ready
controller.ready(() => {
  // ...

  // START: slash commands
  controller.on('slash_command', async (bot, message) => {
    const numCats = parseInt(message.text);
    const response = makeCatMessage(numCats);
    bot.httpBody({ text: response });
  });
  // END: slash commands
});

// ...
```

馬上試試

輸入「/cats」加上數字，即可傳送斜線指令。例如： `/cats 8`

/cats 8

⚡ | **B** *I* `</>` [🔗](#) $\frac{1}{2}$ $\frac{1}{3}$ $\frac{1}{4}$ $\frac{1}{5}$ $\frac{1}{6}$

機器人會回覆 8 隻貓的圖片，只有你能看到：

👁 Only visible to you



Kittenbot APP 5:05 PM



Message #codelab

⚡ | **B** *I* `</>` [🔗](#) $\frac{1}{2}$ $\frac{1}{3}$ $\frac{1}{4}$ $\frac{1}{5}$ $\frac{1}{6}$

12. 清除所用資源

恭喜！您現在已在 Cloud Run 上執行 Slack 機器人。現在要清除使用的資源 (節省成本，當個優質的雲端使用者)。

刪除專案

您可以直接從 Cloud Shell 刪除整個專案：

```
gcloud projects delete $PROJECT_ID
```

或者，如果您想逐一刪除不同資源，請繼續閱讀下一節。

刪除部署作業

```
gcloud run services delete kittenbot --region $REGION
```

指令輸出

```
Service [kittenbot] will be deleted.  
Do you want to continue (Y/n)? y  
Deleted service [kittenbot].
```

刪除用戶端簽署密鑰

```
gcloud secrets delete client-signing-secret
```

指令輸出

```
You are about to destroy the secret [client-signing-secret] and its  
[1] version(s). This action cannot be reversed.  
Do you want to continue (Y/n)? y  
Deleted secret [client-signing-secret].
```

刪除機器人權杖密鑰

```
gcloud secrets delete bot-token
```

指令輸出

```
You are about to destroy the secret [bot-token] and its [1]  
version(s). This action cannot be reversed.  
Do you want to continue (Y/n)? y  
Deleted secret [bot-token].
```


刪除儲存空間 bucket

首先，列出 Google Cloud Storage bucket，取得 bucket 路徑：

```
gsutil ls
```

指令輸出

```
gs://[REGION.]artifacts.<PROJECT_ID>.appspot.com/  
gs://<PROJECT_ID>_cloudbuild/
```

現在，請刪除構件 bucket：

```
gsutil rm -r gs://[REGION.]artifacts.${PROJECT_ID}.appspot.com/
```

指令輸出

```
Removing gs://[REGION.]artifacts.<PROJECT_ID>.appspot.com/...
```

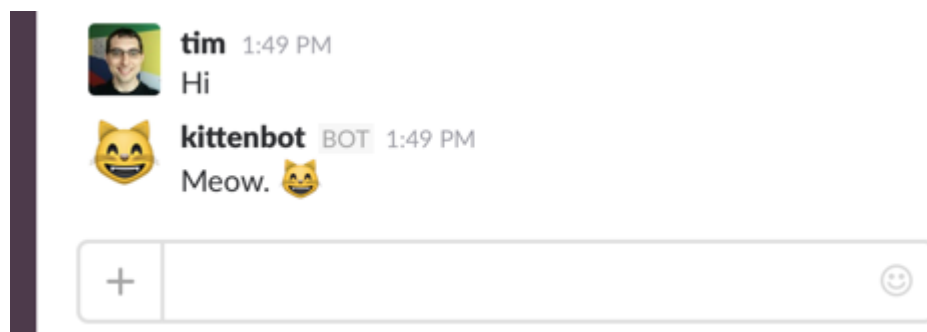
最後，刪除 cloudbuild bucket：

```
gsutil rm -r gs://${PROJECT_ID}_cloudbuild/
```

指令輸出

```
Removing gs://<PROJECT_ID>_cloudbuild/...
```

13. 恭喜！



您現在已瞭解如何在 Cloud Run 上執行 Slack 機器人！

我們目前只介紹了部分這項技術，建議您進一步探索自己的 Cloud Run 部署作業。

涵蓋內容

- 在 Slack 中建立機器人自訂整合
- 使用 Secret Manager 保護 Slack 密鑰
- 在 Cloud Run 上部署 Slack 機器人

後續步驟

- 完成更多 [Cloud Run 教學課程](#)

瞭解詳情

- 前往 GitHub 查看其他 [Google Cloud Slack 整合範例](#)。
 - 自行試用其他 Google Cloud 功能，並參考我們的[教學課程](#)。
-